

TraceX — Digital Incident Investigation & Evidence Management System

Complete Architecture & Implementation Blueprint (Source of Truth for Antigravity Build)

1. Executive Summary

TraceX is a locally-run web application that lets a student “investigator” open a case, register digital evidence, hash and verify it, track a chain of custody, import synthetic security logs, reconstruct an incident timeline, correlate evidence/events with explainable rules, and generate a structured investigation report. It is built with **Flask + SQLite + server-rendered HTML/CSS/JS**, requires no cloud services or ML, and every “finding” the system produces is explicitly labeled by evidentiary strength (Observed Evidence → Detected Indicator → System-Generated Finding → never a “Confirmed Fact”). This blueprint is the frozen source of truth for building it in Antigravity, phase by phase.

2. Critical Evaluation of the Idea

CS relevance: Strong — normalized relational schema, layered architecture, algorithmic timeline sorting, rule-based correlation, hashing, REST API design, OOP service classes.

Cybersecurity relevance: Strong — SHA-256 integrity verification, auth with hashed passwords, RBAC, input/file validation, log-based indicator detection, deterministic severity scoring.

Digital forensics relevance: Central, not decorative — evidence acquisition metadata, chain-of-custody (append-only), integrity verification workflow, timeline reconstruction, correlation, and a report that separates fact from inference. These are the actual pillars of real DFIR tooling (conceptually mirroring EnCase/Autopsy custody & hashing workflows), just scoped to synthetic data.

Beyond CRUD: A CRUD app stores and displays records. TraceX additionally (a) computes and re-verifies cryptographic hashes and changes state based on the result, (b) enforces an append-only audit trail rather than editable rows, (c) runs a deterministic rule engine over log events to produce severity-scored indicators, and (d) correlates independent entities (evidence, events, users, timestamps) into a graph the investigator can read.

Beyond a log viewer: Logs aren’t just displayed — they’re parsed into a typed event model, scored by rules, and stitched into a timeline that cross-references evidence.

Live demo suitability: Yes — a 10-minute scripted scenario (Section 30) exercises every module visibly (create case → add evidence → hash → tamper → re-verify → import logs → see indicators → see timeline → see correlation → generate report).

Viva explainability: Every algorithm (hashing, timeline sort, rule matching, correlation) is deterministic and can be explained on a whiteboard in under two minutes — no black-box ML to defend.

Weak spots fixed: The original brief over-lists optional entities/pages; this blueprint trims to a MUST-HAVE core (Section 38) so the project stays buildable in the available time, with SHOULD/OPTIONAL layered on top only if time remains.

3. Final Project Definition

3.1 Title: TraceX — Digital Incident Investigation & Evidence Management System **3.2 Tagline:** “From evidence to timeline — a transparent forensic investigation workspace.” **3.3 One-paragraph overview:** TraceX is a local web application for managing simulated digital-forensics investigations: it registers evidence with cryptographic hashes, tracks an append-only chain of custody, parses synthetic security logs into a searchable event timeline, applies deterministic rules to flag suspicious patterns, correlates related evidence and events, and produces a structured, auditable investigation report — all using safe synthetic data.

3.4 Detailed overview: An investigator (role: ADMIN or INVESTIGATOR) logs in, opens a case tied to a simulated incident, and registers evidence items (log bundles, “files”, memory-dump stand-ins — all synthetic). Each item is SHA-256 hashed on ingestion; the investigator can re-verify at any time, and the system flags VERIFIED / MODIFIED / MISSING. Every action on evidence writes an immutable custody record. Synthetic logs are imported and parsed into typed SecurityEvent rows; a rule engine scans them for patterns (repeated failed logins, permission changes after file access, etc.) and emits severity-scored Finding records. A correlation engine links events/evidence sharing a user, timestamp window, or hash. The dashboard aggregates case/evidence/finding counts and the timeline; a report generator compiles everything into one document, clearly tagging system output vs. investigator input.

3.5 Problem Statement: Student-level forensic exercises rarely have tooling that ties evidence integrity, custody, log analysis, and timeline reconstruction into one coherent, explainable workflow — most course projects either fake it with a static UI or over-reach into real offensive tooling. TraceX fills that gap safely.

3.6 Motivation: Build hands-on understanding of core DFIR concepts (custody, hashing, timeline, correlation) without needing proprietary tools, real incidents, or unsafe data.

3.7 Objectives: 1. Implement verifiable evidence integrity via SHA-256. 2. Implement an auditable, append-only chain of custody. 3. Parse synthetic logs into a normalized event model. 4. Reconstruct a chronological incident timeline. 5. Apply deterministic, explainable indicator/correlation rules. 6. Produce a structured investigation report distinguishing fact from inference. 7. Demonstrate secure application design (auth, validation, safe file handling).

3.8 Target Users: Cybersecurity/forensics students, academic evaluators, viva panel — anyone needing a teaching-scale investigation tool.

3.9 Scope: Single-investigator or small-team (ADMIN/INVESTIGATOR) case management; synthetic evidence and logs only; local deployment.

3.10 Out of Scope: Real malware execution, live network capture/intrusion detection, cloud storage, multi-tenant SaaS, ML-based anomaly detection, mobile/app forensics, actual file-system/memory acquisition.

3.11 Innovation: See Section 37.

3.12 Academic relevance: Touches DBMS (schema/normalization), OOP (service classes), DSA (sorting/graph-like correlation), Software Engineering (layered architecture, testing), Security (hashing, auth, validation) — matching a Cybersecurity & Digital Forensics specialization directly.

3.13 Real-world relevance: Mirrors genuine DFIR practice concepts (custody chains, hash verification, timeline analysis, indicator triage) used in real investigations, at a safe educational scale.

3.14 Limitations: Not a legal-grade forensic tool; no real acquisition hardware/write-blockers; rule engine is intentionally simple/deterministic, not ML-based; single-machine, non-distributed.

4-5. Requirements & Functional Modules

Non-Functional Requirements

1. **Usability** — clear role-appropriate dashboard, empty/error states on every page.
2. **Security** — hashed passwords (bcrypt/werkzeug), parameterized queries, file-type/size validation, path-traversal-safe file handling, CSRF tokens on forms.
3. **Reliability** — append-only custody log; DB transactions wrap multi-step writes.
4. **Performance** — timeline/correlation queries indexed on `case_id`, `timestamp`.
5. **Maintainability** — layered architecture, one responsibility per service class.
6. **Portability** — SQLite file DB, runs with `python app.py`, no external services.

Functional Modules (final six)

#	Module	Core Responsibility
1	Case / Investigation Management	Create/update/close/view cases
2	Digital Evidence Management	Register evidence + SHA-256 hashing
3	Chain of Custody	Append-only action log per evidence item
4	Log Analysis & Timeline	Parse synthetic logs → events → sorted timeline
5	Correlation & Dashboard	Rule-based linking + aggregate view
6	Investigation Reporting	Compile case into a labeled report

6. Input / Processing / Output Table (per module)

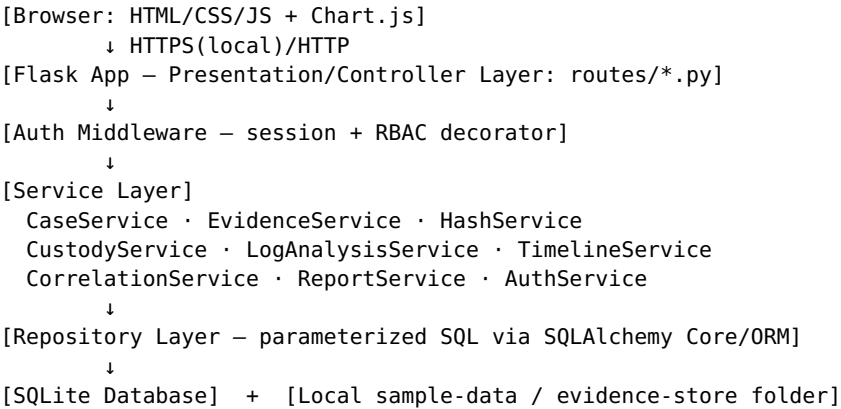
Module	Input	Processing
Case Mgmt	title, incident_type, description, investigator	validate → insert Case row, set status=OPEN
Evidence	file (synthetic), name, type, source, description, case_id	validate file type/size → compute SHA-256 Evidence + initial custody record
Hash Verify	evidence_id	re-read stored file → recompute hash → compare to stored hash
Custody	evidence_id, action, actor, remarks	validate action enum → append custody record (create/update/delete)
Log Import	.csv/.jsonl synthetic log file, case_id	validate schema → parse rows → normalize insert SecurityEvent rows
Log Analysis	case_id	run rule set over that case's events
Timeline	case_id	fetch events+findings → sort by timestamp
Correlation	case_id	apply correlation keys (user, time-window) over events/evidence
Report	case_id	aggregate case+evidence+custody+timeline+findings

7. Complete User Workflow

Login → Dashboard → Create Case → Add Evidence → Generate SHA-256 Hash
 → Store Evidence Metadata → Chain-of-Custody entry (auto on each action)
 → Import Synthetic Logs → Parse Events → Run Rule Engine (Findings)
 → Generate Timeline → Run Correlation Engine → Review Findings
 → Generate Report → (optionally) Close Investigation

Every arrow above is a real state transition in the DB (not just UI navigation): case status, evidence integrity status, and custody history all update as the investigator moves through the flow.

8. System Architecture



Each service is a plain Python class with injected repository access — no service talks to another service’s repository directly, only through the service’s own public methods (keeps coupling low, easy to unit test and explain in viva).

9. Technology Stack

Criterion	A: Flask+SQLite+HTML/JS	B: FastAPI+SQL
Beginner friendliness	High	Medium
Matches Zade’s existing toolset (Python/Flask)	✓	Partial
Setup/demo simplicity	Very high (single process)	Medium (2 pr
DB integration/normalization	Easy (SQL, real FKs)	Easy
Viva explainability	Very high	High

Final choice: Option A — Flask + SQLite + server-rendered HTML/CSS + vanilla JS/Chart.js. It’s local, free, single-process, needs no build tooling, and directly reuses skills already in the toolset. Optional: Docker Compose wrapper for a one-command run, since Docker is already a known tool.

10. Database Design — ARCHITECTURE FREEZE – DATABASE SCHEMA

Entities (final six — trimmed from the original nine)

User, Case, Evidence, CustodyRecord, SecurityEvent, Finding (covers both rule-matched indicators and correlation results via a type column, avoiding a redundant separate Correlation table while keeping the same information).

Entity	Key Attributes
--------	----------------

User	id, username (UNIQUE), password_hash, role (ADMIN/INVESTIGATOR), created_at
Case	id, title, incident_type, description, status (OPEN/CLOSED), investigator_id, created_at, closed_at
Evidence	id, case_id, name, type, source, file_path, file_size, sha256_original, sha256_current, integrity_status (VERIFIED/MODIFIED/MISSING/NOT_VERIFIED), collected_at
CustodyRecord	id, evidence_id, action (COLLECTED/TRANSFERRED/REVIEWED/VERIFIED), actor_id, prev_status, new_status, remarks, timestamp
SecurityEvent	id, case_id, event_type, event_user, source, ip_ref, event_timestamp, raw_line
Finding	id, case_id, kind (INDICATOR/CORRELATION), rule_severity (INFO/LOW/MEDIUM/HIGH/CRITICAL), related_evidence_ids (JSON), related_evidence_ids (JSON), description, created_at

Constraints: Case.title NOT NULL; Evidence.sha256_original NOT NULL after upload; CustodyRecord has no update endpoint (enforced at API layer, not just DB); CHECK on Finding.severity IN allowed set; CHECK on Evidence.integrity_status IN allowed set.

Normalization: 3NF — no repeating groups, all non-key attributes depend only on the PK (e.g., custody history is its own table rather than columns bolted onto Evidence).

Indexing: index on Evidence.case_id, SecurityEvent.case_id + event_timestamp, Finding.case_id — these are the columns every dashboard/timeline/correlation query filters or sorts on.

Sample record (Evidence): {id:1, case_id:1, name:"auth.log", type:"LOG", source:"synthetic-siem", sha256_original:"9f86d0...", integrity_status:"VERIFIED"}

ARCHITECTURE FREEZE — DATABASE SCHEMA. Any change during Antigravity build must be explicitly justified and flagged before being made.

11-13. Forensics, Hashing & Custody Design

Forensic concepts implemented: evidence identification (registering an item), acquisition metadata (source/collected_at), integrity verification (hashing), custody (who touched what, when), timeline reconstruction, rule-based correlation, and labeled reporting.

What it does NOT do: real acquisition (write-blocking, disk imaging), legal chain-of-custody enforcement, or definitive attribution.

Hashing flow: upload → SHA-256 computed → stored as sha256_original → on “Verify,” file is re-hashed into sha256_current → compared → status set to VERIFIED/MODIFIED/MISSING/NOT_VERIFIED. A hash proves *the bytes are unchanged since acquisition*, never who touched them.

Custody design: every state-changing action on evidence (add, verify, review, transfer, export) inserts a new CustodyRecord row capturing prev/new status and actor — rows are never edited or deleted, so the full history is always reconstructable, which is the actual point of a custody chain (tamper-evidence over time, not just current state).

14-17. Log Analysis, Timeline, Correlation, Severity

Supported synthetic log format: CSV/JSONL with columns timestamp, event_type, user, source, ip_ref. Event types: LOGIN_SUCCESS, LOGIN_FAILED, MULTIPLE_LOGIN_FAILURES, FILE_CREATED, FILE_MODIFIED, FILE_DELETED, ACCOUNT_LOCKED, UNUSUAL_ACCESS, PERMISSION_CHANGED.

Rule engine (deterministic, no ML): - R1: ≥ 5 LOGIN_FAILED for same user within 10 min $\rightarrow$ INDICATOR, severity MEDIUM, label "Potentially suspicious authentication activity." - R2: ACCOUNT_LOCKED following R1 $\rightarrow$ severity HIGH, "Security indicator: account lock after repeated failures." - R3: UNUSUAL_ACCESS then PERMISSION_CHANGED for same user within 15 min $\rightarrow$ severity HIGH, "Rule matched: suspicious access-then-privilege-change sequence." - R4: Evidence.integrity_status = MODIFIED $\rightarrow$ severity CRITICAL, "Integrity warning: registered evidence changed after acquisition."

Timeline algorithm: fetch case's SecurityEvent + Finding rows $\rightarrow$ normalize timestamps to UTC $\rightarrow$ sort by timestamp (Python sorted, $O(n \log n)$) $\rightarrow$ group by day for display. Explainable in one sentence for viva.

Correlation engine: IF two events/evidence share case_id AND (same user OR same source OR same hash) AND occur within a bounded time window THEN insert Finding(kind=CORRELATION). Always phrased as "potentially related," never causal.

Severity model: INFO (informational match) < LOW < MEDIUM (single rule match) < HIGH (chained rule match) < CRITICAL (integrity violation) — fixed, deterministic mapping per rule, no scoring formula to obscure explainability.

18-19. Security & Role Design

- Auth: username/password, hashed with Werkzeug's generate_password_hash (bcrypt-backed).
 - Session-based auth (Flask session, HttpOnly + Secure cookie flags).
 - RBAC via a @role_required(...) decorator: **ADMIN** can manage users and close/delete cases; **INVESTIGATOR** can create/edit cases and all evidence/log/report actions but not manage users.
 - Input validation on every form (server-side, not just client-side).
 - File validation: extension allow-list, max size, filename sanitized (werkzeug.utils.secure_filename), stored outside any web-servable path, no path built from raw user input.
 - SQL injection prevented via parameterized queries/ORM only — no string-built SQL.
 - CSRF tokens on all state-changing forms (Flask-WTF).
 - Generic error messages to the client; full stack traces only to server logs.
-

20. UI / UX Design (page list)

Login · Dashboard (case/evidence/finding counts, integrity mismatches, recent timeline) · Cases (list+filter) · Case Detail (tabs: Evidence / Custody / Logs / Timeline / Findings) · Add Evidence form · Evidence Detail (hash, verify button, custody history) · Log Import form · Timeline view (chronological cards) · Findings view (severity-colored list) · Report view/download · (Admin-only) User management.

Each page needs an explicit empty state (“No evidence yet — add your first item”) and error state (toast/banner), consistent with the “clean, practical” preference — no dead-end blank screens.

21. Class / Component Design (final)

Models: User, Case, Evidence, CustodyRecord, SecurityEvent, Finding

Services: AuthService, CaseService, EvidenceService, HashService, CustodyService, LogAnalysisService, TimelineService, CorrelationService, ReportService
Controllers (Flask blueprints): auth_routes, case_routes, evidence_routes, log_routes, timeline_routes, report_routes

Example — EvidenceService: depends on EvidenceRepository, HashService, CustodyService; methods add_evidence(), verify_evidence(), get_evidence(case_id). No controller touches a repository directly — only through its service.

22. UML Specifications (textual)

Use Case Diagram actors: Investigator, Admin. Use cases: Manage Case, Manage Evidence, Verify Integrity, Manage Custody, Import Logs, Analyze Logs, View Timeline, View Correlations, Generate Report, Manage Users (Admin only).

Class Diagram: Case 1—* Evidence 1—* CustodyRecord; Case 1—* SecurityEvent; Case 1—* Finding; User 1—* Case (as investigator); User 1—* CustodyRecord (as actor).

Sequence — Add Evidence: Investigator → EvidenceController → EvidenceService.add_evidence() → HashService.compute_sha256() → EvidenceRepository.save() → CustodyService.log(action=COLLECTED) → response.

Sequence — Verify Integrity: Investigator → EvidenceController → EvidenceService.verify_evidence() → HashService.recompute() → compare → EvidenceRepository.update_status() → CustodyService.log(action=VERIFIED).

Sequence — Analyze Logs: Investigator → LogController → LogAnalysisService.import() → parse rows → SecurityEventRepository.bulk_insert() → LogAnalysisService.run_rules() → FindingRepository.bulk_insert().

Sequence — Generate Timeline: Investigator → TimelineController → TimelineService.build(case_id) → fetch events+findings → sort → return ordered view.

23. Folder Structure (final)

```
TraceX/
├─ app.py
├─ config.py
├─ requirements.txt
├─ .env.example
├─ .gitignore
├─ README.md
├─ statement.md
├─ models/
│   └─ user.py  case.py  evidence.py
```

```

|   |— custody_record.py  security_event.py  finding.py
|— services/
|   |— auth_service.py  case_service.py  evidence_service.py
|   |— hash_service.py  custody_service.py  log_analysis_service.py
|   |— timeline_service.py  correlation_service.py
report_service.py
|— repositories/
|   |— case_repository.py  evidence_repository.py
custody_repository.py
|   |— event_repository.py  finding_repository.py
user_repository.py
|— routes/
|   |— auth_routes.py  case_routes.py  evidence_routes.py
|   |— log_routes.py  timeline_routes.py  report_routes.py
|— templates/          (Jinja2 HTML per page listed in §20)
|— static/css/  static/js/  (Chart.js dashboard + timeline JS)
|— database/
|   |— schema.sql  seed.py  tracex.db (generated)
|— sample-data/
|   |— logs_normal.csv  logs_bruteforce.csv
logs_privilege_change.csv
|   |— evidence_samples/
|— evidence-store/      (uploaded synthetic evidence files, not
web-servable)
|— tests/
|   |— test_hash_service.py  test_custody_service.py
|   |— test_log_analysis.py  test_timeline.py  test_correlation.py
|— docs/
|   |— architecture.md  er-diagram.png  uml/
|— reports/             (generated report output)

```

evidence-store/ must never be placed under static/ (would make raw evidence web-downloadable without auth). database/tracex.db is generated, never hand-edited.

24. File Responsibility Map (excerpt — full map to be maintained in docs/)

File	Responsibility	Depends (
services/hash_service.py	compute/recompute SHA-256	hashlib
services/evidence_service.py	evidence lifecycle	hash_service, custody_servic evidence_repo
services/log_analysis_service.py	parse logs + run rules	event_reposi finding_reposi
services/timeline_service.py	sort/group events for display	event_reposi finding_reposi
services/correlation_service.py	rule-based linking	event_reposi evidence_repo finding_reposi
services/report_service.py	assemble final report	all repositorie

25. API Design (REST, session-authenticated)

Method	Endpoint	Purpose	Auth
POST	/auth/login	login	none
POST	/cases	create case	INVESTIGATOR+
GET	/cases / /cases/{id}	list/view case	any logged in
PUT	/cases/{id}	update case	owner/ADMIN
POST	/cases/{id}/close	close case	owner/ADMIN
POST	/evidence	add evidence	INVESTIGATOR+
GET	/evidence/{id}	view evidence	any logged in
POST	/evidence/{id}/verify	re-hash & verify	INVESTIGATOR+
GET	/evidence/{id}/custody	custody history	any logged in
POST	/logs/import	import log file	INVESTIGATOR+
POST	/logs/{case_id}/analyze	run rule engine	INVESTIGATOR+
GET	/timeline/{case_id}	timeline view	any logged in
GET	/correlations/{case_id}	correlation results	any logged in
POST	/reports/{case_id}	generate report	INVESTIGATOR+

Each POST/PUT validates body server-side and returns 4xx with a field-level error map on failure.

26. Error Handling Strategy

Central Flask error handlers map: 400 invalid/missing input, 401 unauthenticated, 403 wrong role, 404 missing case/evidence, 409 duplicate/conflicting action, 413 file too large, 415 bad file type, 422 malformed log rows (partial import allowed, bad rows reported), 500 unexpected — logged server-side with a generic user-facing message.

27. Testing Strategy

Unit tests per service (hash correctness, custody append-only behavior, rule matching, timeline ordering, correlation matching); integration tests per route (auth-gated access, end-to-end add→verify→tamper→re-verify). Sample table:

Test ID	Module	Input	Expected	Status
T01	Hash	valid file	VERIFIED after verify	Pass
T02	Hash	tampered file	MODIFIED after verify	Pass
T03	Custody	verify action	new append-only row, old rows unchanged	Pass
T04	Log Import	malformed row	row skipped, rest imported	Pass

T05	Rules	5 failed logins in window	MEDIUM indicator created	Pass
T06	Auth	wrong role on /cases POST	403	Pass
T07	Timeline	mixed events	strictly chronological output	Pass

28. Safe Demonstration Data & Scenario

Five synthetic CSV scenarios (normal auth, brute-force, privilege-change sequence, tamper case, multi-event correlated timeline) ship in sample-data/. **10-minute demo:** login → create case → add evidence + hash → import logs_bruteforce.csv → run analysis → show MEDIUM/HIGH findings → show timeline → show correlation → tamper the evidence file on disk → re-verify → show MODIFIED + new CRITICAL finding → generate report.

29. Antigravity Development Roadmap (16 phases)

Phase	Objective	Key Files	C
0	Freeze architecture/schema	docs/architecture.md	re co
1	Init Flask project, config, .env	app.py, config.py, requirements.txt	ap
2	DB schema + seed	database/schema.sql, seed.py	ta cr
3	Auth (login, hashing, RBAC)	auth_service.py, auth_routes.py	lo
4	Case management	case_service/repository/routes	CI wo
5	Evidence upload	evidence_service/repository/routes	ev st
6	Hashing & verify	hash_service.py	ve st
7	Custody	custody_service/repository	ap hi: vi:
8	Log import/parse	log_analysis_service.py (parse)	ev in:
9	Rule engine	log_analysis_service.py (rules)	fir cr
10	Timeline	timeline_service/routes	so re
11	Correlation	correlation_service.py	co fir cr
12	Dashboard	templates/dashboard.html, JS	co re
13	Report generation	report_service/routes	re dc
14	Security hardening	validation, CSRF, file safety	ch

	pass		sa
15	Testing	tests/*	all gr
16	Docs + GitHub polish	README, statement.md, docs/	re re

Every phase prompt for Antigravity must include: *“Do not change the approved architecture, database schema, API contracts, or existing working modules unless the requested implementation is impossible without a change. If a change appears necessary, STOP and explain the proposed change before applying it.”*

30. Phase-wise Antigravity Prompts (copy-paste ready)

Phase 1 prompt:

Project: TraceX (Flask + SQLite). Architecture frozen – see docs/architecture.md.
Objective: Initialize the project skeleton only.
Create: app.py, config.py, requirements.txt, .env.example, .gitignore,
folders models/ services/ repositories/ routes/ templates/ static/ database/
sample-data/ evidence-store/ tests/ docs/ reports/ exactly as specified.
Do NOT implement any business logic yet – stub files only.
Do not change the approved architecture, database schema, API contracts, or
existing working modules unless the requested implementation is impossible
without a change. If a change appears necessary, STOP and explain the
proposed change before applying it.
Test: `python app.py` starts a bare Flask server without errors.
Report what was created and confirm phase completion.

Phase 2 prompt:

Current state: Phase 1 complete, Flask skeleton running.
Objective: Implement the frozen database schema exactly as in docs/architecture.md
Section 10 (User, Case, Evidence, CustodyRecord, SecurityEvent, Finding).
Create: database/schema.sql, database/seed.py, models/*.py (ORM classes matching
the schema 1:1, no extra fields).
Do not add, remove, or rename any table/column from the frozen schema.
Do not change the approved architecture, database schema, API contracts, or
existing working modules unless the requested implementation is impossible
without a change. If a change appears necessary, STOP and explain the
proposed change before applying it.
Test: running seed.py creates tracex.db with all six tables and one seed admin user.
Report what was created and confirm phase completion.

(Phases 3–16 follow the identical template: state current status → objective scoped to exactly that phase’s files from Section 29 → the same “do not change frozen architecture” clause → a concrete test checkpoint → report + confirmation. Generate each remaining phase prompt this same way when you reach it, substituting that phase’s files/objective/test from the table in Section 29.)

31. GitHub Structure

README.md (overview, problem statement, features, stack, architecture diagram, install/run steps, test steps, screenshots, sample data, limitations, future work), statement.md (problem statement, scope, target users, high-level features), .gitignore (venv, *.db, .env, __pycache__), .env.example (no real secrets), docs/ (architecture.md, ER diagram, UML). No .env, no real credentials, no real personal data ever committed.

32. Project Report Structure

Cover Page → Introduction → Problem Statement → Objectives → Functional/Non-functional Requirements → System Architecture (diagram from §8) → Design Diagrams (UML from §22) → Database Design (§10, with ER diagram) → Design Decisions & Rationale (why Flask/SQLite, why rule-based not ML) → Implementation Details (per module, from §29 phases) → Screenshots/Results (from the demo scenario, §28) → Testing Approach (§27) → Challenges Faced → Learnings & Key Takeaways → Future Enhancements (from §36 SHOULD/OPTIONAL) → References.

33. Viva Questions & Answers (25)

1. **Why TraceX?** — It teaches the full DFIR lifecycle (evidence→custody→analysis→report) in one coherent, safe tool.
2. **Why digital forensics?** — Directly matches the Cybersecurity & Digital Forensics specialization and has real-world analogues.
3. **Why SHA-256?** — Collision-resistant, standard for forensic integrity checks; any bit change produces a completely different hash.
4. **What is chain of custody?** — An auditable, append-only record of who did what to evidence and when, so its history can’t be silently rewritten.
5. **Why a relational database?** — Cases/evidence/events/findings have clear foreign-key relationships; normalization avoids duplicate/inconsistent data.
6. **Why this layered architecture?** — Separates presentation, business logic, and data access so each can be tested/changed independently.
7. **Why rule-based, not ML, for log analysis?** — Deterministic, fully explainable, no training data needed, appropriate for a bounded academic scope.
8. **How does timeline reconstruction work?** — Fetch events+findings for a case, normalize timestamps, sort $O(n \log n)$, group by day.
9. **How does correlation work?** — Matches events/evidence sharing case, user, source, or hash within a bounded time window; never claims causation.
10. **What happens when evidence is modified?** — Re-hash mismatches the original; status flips to MODIFIED and a CRITICAL finding is raised.

11. **How is authentication secured?** — Hashed passwords (Werkzeug/bcrypt), session cookies, RBAC decorator on routes.
 12. **What are the limitations?** — Not legally admissible tooling, synthetic data only, single-machine, deterministic rules only.
 13. **What makes it innovative?** — See Section 34 — integrated, explainable pipeline rather than any single isolated feature.
 14. **What CS concepts are used?** — Sorting, normalized relational modeling, layered OOP design, REST API design.
 15. **What cybersecurity concepts are used?** — Hashing/integrity, RBAC, input/file validation, secure session handling.
 16. **Why append-only custody records instead of editable rows?** — Mutable history defeats the purpose of a custody trail; forensic soundness requires tamper-evidence.
 17. **What does a hash prove and not prove?** — Proves byte-for-byte integrity since a point in time; never proves *who* changed something.
 18. **Why SQLite over a server DB?** — Zero-setup, file-based, perfectly adequate at this scale, keeps the demo self-contained.
 19. **How would you scale this to multiple investigators?** — Move to Postgres, add per-case access control beyond role, add optimistic locking on cases.
 20. **How do you prevent SQL injection?** — All queries parameterized/ORM-based, never string-concatenated.
 21. **How do you prevent path traversal on file upload?** — `secure_filename`, fixed storage directory, extension allow-list, no user-controlled paths.
 22. **What's the severity model based on?** — A fixed, documented mapping from rule/violation type to severity — no opaque scoring.
 23. **How is "finding" different from "fact"?** — Findings are system-generated inferences from rules; the report explicitly labels them as such, never as confirmed conclusions.
 24. **What would you add with more time?** — See Section 36 SHOULD/OPTIONAL list.
 25. **Why not use a NoSQL database?** — The data is inherently relational (FKs, joins for timeline/correlation); a relational DB models it more naturally and enforces integrity constraints.
-

34. Innovation (concrete, not buzzwordy)

- Integrating evidence integrity + append-only custody + rule-based log analysis + explainable correlation + labeled reporting into **one coherent pipeline**, rather than any single isolated feature.
 - Explicit **evidentiary-strength labeling** (Observed / Indicator / Finding / never Fact) baked into the data model and report — a design choice, not just UI copy.
 - **Explainable correlation engine** using named, bounded rules instead of a black box.
 - (*Optional, not mandatory*) A visual evidence↔event↔user relationship graph on the dashboard.
-

35. Complexity Control

MUST HAVE: Modules 1-4 fully working (Case, Evidence+Hash, Custody, Log+Timeline), auth+RBAC, SQLite schema as frozen, basic dashboard, core tests. **SHOULD HAVE:** Correlation engine, report generation/export, severity-colored findings UI, seeded demo data script. **OPTIONAL:** Relationship graph visualization, PDF report export, Docker Compose wrapper, multi-case comparison view.

Development priority: 1) Core functionality → 2) Security → 3) UI/UX → 4) Visualization → 5) Reporting → 6) Optional enhancements.

36. VERSION 1.0 — ARCHITECTURE FREEZE DOCUMENT

- **Stack:** Flask (Python) + SQLite + Jinja2/HTML/CSS + vanilla JS/Chart.js.
- **Architecture:** Browser → Flask routes → Auth middleware (RBAC) → Service layer → Repository layer → SQLite + local evidence-store.
- **Modules:** Case Management, Evidence Management, Chain of Custody, Log Analysis & Timeline, Correlation & Dashboard, Reporting.
- **Folder structure:** as in Section 23.
- **DB entities/relations:** User, Case, Evidence, CustodyRecord, SecurityEvent, Finding — relationships as in Section 10 ER model; schema frozen, changes require explicit justification.
- **API contracts:** as in Section 25.
- **Class/component list:** as in Section 21.
- **Security design:** hashed passwords, session auth, RBAC, parameterized queries, validated/sanitized file handling, CSRF protection.
- **Workflow:** as in Section 7.

37. Final Implementation Checklist

- ☐ Architecture & schema reviewed and frozen (Phase 0)
- ☐ Project skeleton boots (Phase 1)
- ☐ Schema + seed data created (Phase 2)
- ☐ Auth + RBAC working (Phase 3)
- ☐ Case CRUD working (Phase 4)
- ☐ Evidence upload + hashing working (Phases 5-6)
- ☐ Custody trail append-only and visible (Phase 7)
- ☐ Log import + rule engine producing findings (Phases 8-9)
- ☐ Timeline sorted and rendering (Phase 10)
- ☐ Correlation engine producing linked findings (Phase 11)
- ☐ Dashboard aggregates render correctly (Phase 12)
- ☐ Report generation working (Phase 13)
- ☐ Security checklist (§18) satisfied (Phase 14)
- ☐ All tests green (Phase 15)
- ☐ README/statement/docs complete, repo demo-ready (Phase 16)